OFFICIAL CLINICAL AI DOCUMENT

Project README - MedReporter AI

🫁 Automated Medical Image Reporting Using AI

An end-to-end, clinical-grade AI platform that generates structured radiology reports from medical images (Chest X-Rays, CT Scans, MRIs) with Explainable AI (XAI), PACS viewing tools, longitudinal prior study comparison, patient-friendly translation, and an interactive AI Radiologist Copilot.

🌟 Key Features

1. Vision-Language AI Engine

- **Salesforce BLIP Foundation Model**: Pre-trained vision-language model generating structured **Findings** and clinical **Impression**.
- **Calculated Confidence Ring**: Sequence log-probability scoring displaying real model certainty (High / Medium / Low).
- **Triage & Urgency Classification**: Automated severity tagging (`Normal`, `Attention Required`, `STAT Urgent Emergency`).

2. Clinical PACS Diagnostic Viewer

- **DICOM-Grade Windowing**: Sliders and one-click presets for Soft Tissue, Bone Window, Lung Window, and Brain CT.
- **Interactive Manipulation**: Smooth zoom (up to 4x), pan, 8-bit negative inversion, and calibration reticle grid.
- **Explainable AI (XAI) Heatmaps**: Visual attention hot zones pinpointing suspected infiltrates, pneumothorax, or cardiomegaly with live crosshair ROI targeting.

3. Longitudinal Prior Study Comparison

- **Dual Synchronized PACS**: Compare current scan against historical baseline studies side-by-side with synchronized zoom and windowing.
- **Interactive Curtain Swipe**: Drag the splitter bar across scans to detect subtle interval changes.
- **Rapid Blink Alternator**: Rapidly toggles current vs prior image to immediately reveal newly developed lesions.
- **AI Interval Delta Summary**: Automatically details whether findings are *Progressed*, *Unchanged*, or *Resolved*.

4. Patient-Friendly Translation & Copilot

- **Empathetic Layperson Translator**: Translates dense radiology jargon into 6th-grade reading level summaries with key takeaways and questions to ask the doctor.
- **Interactive AI Radiologist Copilot**: Chat assistant for clinicians discussing ACR Appropriateness Criteria®, Fleischner Society guidelines, and differential diagnoses.
- **Searchable Medical Glossary**: 15+ interactive definitions with plain-English explanations and phonetic pronunciations.

5. Physician Sign-Off & Multi-Format Export

- **Clinical Sign-Off**: Radiologist can edit findings, add internal notes, and digitally sign reports with practitioner license and institution stamps.
- **Multi-Format Export**: Download structured plain-text `.txt`, print formatted medical records with hospital letterhead, or export the entire clinical database as a batch `.csv` archive.

📐 System Architecture

```
clgpro/
├── backend/                 # FastAPI Python Backend
│   ├── app/
│   │   ├── main.py          # Application entrypoint & static SPA mounting
│   │   ├── config.py        # Settings & environment variables
│   │   ├── database.py      # Async SQLAlchemy & auto-migration
│   │   ├── api/
```

```
│   │   │   ├── auth.py      # Authentication (JWT + bcrypt)
│   │   │   └── routes.py    # Report CRUD, Copilot, XAI, Glossary, Exports
│   │   ├── models/          # Database ORM & Pydantic schemas
│   │   └── services/        # AI model loader, image processor, report generator
│   ├── uploads/             # Storage directory for uploaded scans
│   ├── test_system.py       # Automated 14-suite verification test
│   └── requirements.txt     # Python backend dependencies
│
├── frontend/                # React 19 + Vite Frontend
│   ├── src/
│   │   ├── pages/           # Dashboard, Upload, Reports, ReportDetail, Profile, Login
│   │   ├── components/      # Sidebar, PACS viewer controls, Glossary modal
│   │   ├── context/         # AuthContext (JWT session management)
│   │   └── api/             # Axios API client
│   └── package.json
│
├── sample_images/           # High-fidelity synthetic radiographs for immediate testing
│   ├── chest_xray_normal.png
│   ├── chest_xray_infiltrate.png
│   └── brain_ct_scan.png
│
├── run_dev.bat              # 1-Click Windows Development Launcher
├── run_fullstack.bat        # 1-Click Windows Unified Fullstack Server (Port 8000)
├── start_backend.sh         # Linux/macOS Backend Launcher
├── start_frontend.sh        # Linux/macOS Frontend Launcher
├── run_fullstack.sh         # Linux/macOS Unified Server Launcher
├── Dockerfile               # Production multi-stage Docker build
├── docker-compose.yml       # 1-Command Docker container deployment
└── render.yaml              # Render cloud deployment blueprint
```

🚀 Quick Start & How to Run

System Prerequisites

- **Python**: 3.10 or 3.11 recommended
- **Node.js**: 18+ (with `npm`)

Method 1: One-Click Development Launchers (Recommended)

On Windows:
Double-click `run_dev.bat` or run in PowerShell:

```
.\run_dev.ps1
```

This automatically launches the **FastAPI backend** on `http://localhost:8000` and the **React Vite frontend** on `http://localhost:5173`.

On Linux / macOS:
Open two terminal windows:

```
# Terminal 1: Backend
chmod +x start_backend.sh
./start_backend.sh

# Terminal 2: Frontend
chmod +x start_frontend.sh
./start_frontend.sh
```

Method 2: Manual Step-by-Step Terminal Execution

1. Setup & Run Backend

```
cd backend

# Create virtual environment
python -m venv venv

# Activate virtual environment
# Windows:
venv\Scripts\activate
# Linux/macOS:
source venv/bin/activate
```

```
# Install dependencies
pip install -r requirements.txt

# Start backend server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Backend API docs will be live at `http://localhost:8000/docs`.

2. Setup & Run Frontend

```
cd frontend

# Install dependencies
npm install

# Start Vite dev server
npm run dev
```

Web application will be live at `http://localhost:5173`.

Method 3: Unified Single-Port Fullstack Server (Port 8000)

To run both the frontend and backend together on a single port (`http://localhost:8000`) without running Vite separately:

On Windows:

```
run_fullstack.bat
```

On Linux / macOS:

```
chmod +x run_fullstack.sh
./run_fullstack.sh
```

FastAPI will serve the production-compiled React application directly at `http://localhost:8000`.

🩺 How to Operate the Application (User Guide)

1. Register & Log In

1. Navigate to `http://localhost:5173` (or `http://localhost:8000`).
2. Click **Create an account** (or use any registered credentials).
3. Fill in your name, email, and password. You will receive an authentication token.

2. Update Practitioner Profile

1. Click **Profile** in the sidebar.
2. Enter your hospital name (e.g., *St. Jude Medical Center*), department (*Diagnostic Radiology*), and license number (*MD-94021*).
3. These details will automatically be stamped on your verified reports.

3. Generate an AI Report

1. Click **Generate Report** in the sidebar.
2. Drag and drop any medical image, or pick one from `sample_images/`:
   - `sample_images/chest_xray_normal.png` (Standard bilateral lung fields)
   - `sample_images/chest_xray_infiltrate.png` (Simulates right lower-lobe consolidation)
   - `sample_images/brain_ct_scan.png` (Axial brain CT slice)
3. Select the modality (**Chest X-Ray**, **CT Scan**, **MRI**, or **Other**).
4. *(Optional)* Click one of the **Clinical Intake Presets** (e.g., *Trauma Intake*, *Infiltrate / Pneumonia*, *Cranial CT*) to auto-populate clinical indications and patient demographics.
5. Click **Generate AI Report**. The system runs inference and navigates to the detailed report page.

4. PACS Diagnostic Viewer Tools

On the report page:

- **Presets**: Click `BONE`, `SOFT TISSUE`, `LUNG`, or `INVERT` to optimize contrast for specific tissue densities.
- **Sliders**: Adjust brightness and contrast manually.
- **Reticle Grid**: Click the grid icon to project an anatomical measurement overlay.

- **Explainable AI (XAI)**: Click **AI Attention Heatmap** to visualize color activation zones where the AI detected abnormalities. Click any zone to lock crosshairs.

5. Prior Study Longitudinal Comparison

1. At the top of the report, locate the **Compare With Prior Study** dropdown.
2. Select an earlier study.
3. Switch between comparison modes:
    - **Dual PACS**: View both studies side-by-side with synchronized zoom and windowing.
    - **Curtain Swipe**: Slide the divider across the scan to detect new lesions.
    - **Blink Mode**: Rapidly flip between studies to identify changes.
4. Read the **AI Interval Summary** for automated progress notes.

6. AI Copilot & Patient Translation

- Click the **Patient Summary** tab to generate a clear, compassionate explanation for patients with suggested questions for their doctor.
- Click the **AI Copilot** tab to query the radiologist assistant for differential diagnoses or clinical guideline references.

7. Sign-Off & Exporting

- Click **Edit / Sign-off** to modify findings or append internal notes.
- Click **Sign & Finalize Report** to apply your official practitioner signature and timestamp.
- Click **Export .TXT** for a plain-text copy, or **Print / PDF** for a printable report with hospital header and signature stamp.
- From the **Report History** page, click **Export All (CSV)** to download a complete research archive of all cases.

🐳 How to Deploy

Option 1: Docker & Docker Compose (Recommended for Production)

Run the entire application in a self-contained container with persistent storage:

```
# Build and run container
docker compose up --build -d

# Check status
docker compose ps

# View logs
docker compose logs -f
```

The application will be accessible at `http://localhost:8000`. Database records and uploaded images are persisted in the `medreporter_data` Docker volume.

To stop the service:

```
docker compose down
```

Option 2: Render Deployment (1-Click Cloud Blueprint)

1. Fork or push this repository to GitHub.
2. Log in to Render.
3. Create a **New Blueprint Instance** and select your repository.
4. Render will read `render.yaml`, configure the persistent 10GB disk for uploads and SQLite, and deploy the application.

Option 3: Railway / Heroku / CapRover

The project contains a `Procfile` configured for container/PaaS platforms:

```
web: cd backend && uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Set the following environment variables in your deployment dashboard:

- `ALLOWED_ORIGINS=["*"]`
- `SECRET_KEY=<your-secret-random-key>`
- `DEVICE=cpu`
- `DATABASE_URL=sqlite+aiosqlite:///./data/medical_reports.db`

Option 4: Linux VPS (Ubuntu / Debian + Systemd)

1. Clone the repository to `/opt/medreporter`.
2. Build the frontend:

```
cd /opt/medreporter/frontend && npm ci && npm run build
```

3. Setup Python venv:

```
cd /opt/medreporter/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Create systemd service `/etc/systemd/system/medreporter.service`:

```
[Unit]
Description=Medical Image Reporter FastAPI Service
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/medreporter/backend
ExecStart=/opt/medreporter/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8
Restart=always

[Install]
WantedBy=multi-user.target
```

5. Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now medreporter
```

🧪 Testing & Verification

The project includes an end-to-end automated verification test suite covering **14 clinical workflows**:

```
# Windows
backend\venv\Scripts\python.exe backend\test_system.py

# Linux/macOS
./backend/venv/bin/python backend/test_system.py
```

Verified Test Suites:

1. Health check & AI model status (`/api/health`)
2. User registration and JWT authentication (`/api/auth/register`)
3. Clinician profile retrieval (`/api/auth/me`)
4. Synthetic radiograph generation & AI report pipeline (`/api/report/generate`)
5. Report detail retrieval with XAI heatmaps & DICOM tags (`/api/reports/{id}`)
6. Uploaded radiograph static media serving (`/api/uploads/{filename}`)
7. Dashboard aggregate statistics and weekly volume (`/api/dashboard`)
8. Full-text search over clinical findings (`/api/reports/search`)
9. Plain-text clinical report export (`/api/reports/{id}/export`)
10. Longitudinal prior study comparison & delta analysis (`/api/reports/{id}/compare/{prior_id}`)
11. Layperson patient summary generation (`/api/reports/{id}/patient-summary`)
12. AI Radiologist Copilot interactive consultation (`/api/reports/{id}/copilot`)
13. System telemetry & hardware metrics (`/api/system/metrics`)
14. Batch clinical research archive export (`/api/reports/export/all`)

⚙️ Environment Variables

Variable	Default	Description
`APP_NAME`	`Medical Image Reporter`	Application display name
`APP_VERSION`	`1.0.0`	Semantic version string
`DEBUG`	`true`	Enable SQL query logs and debug mode
`HOST`	`0.0.0.0`	Server bind IP address
`PORT`	`8000`	Server listen port
`DATABASE_URL`	`sqlite+aiosqlite:///./medical_reports.db`	SQLAlchemy database URL
`SECRET_KEY`	`medical-image-reporter-dev-secret-key-2024`	Secret key for JWT signing
`ALGORITHM`	`HS256`	JWT cryptographic algorithm
`ACCESS_TOKEN_EXPIRE_MINUTES`	`1440` (24h)	Token lifespan
`MODEL_NAME`	`Salesforce/blip-image-captioning-large`	HuggingFace model repo

Variable	Default	Description
`DEVICE`	`cpu`	Inference device (`cpu` or `cuda`)
`UPLOAD_DIR`	`./uploads`	Directory where uploaded scans are stored
`MAX_FILE_SIZE_MB`	`10`	Maximum scan file size in megabytes
`ALLOWED_ORIGINS`	`["*"]`	CORS origin whitelist

🔒 Security & Medical Disclaimer

> **Medical Disclaimer**: This application is built for clinical decision support, academic demonstration, and educational purposes. In real-world hospital environments, reports generated by AI must always be reviewed, edited, and approved by a certified physician or licensed radiologist before clinical intervention.

📄 License

MIT License. Built for advanced medical imaging informatics and AI research.